CS535: Algorithm Design and Analysis

Ordered Dictionaries and Binary Search Trees

Prof. Rohit Pal Singh

Department of Computer Science

University of Wisconsin, Milwaukee

September 30, 2024

Readings: Chapter 3.1

Ordered Dictionaries

- Operations for unordered dictionaries:
  - findElement(k), insertItem(k,e), removeElement(k)

Ordered dictionaries:

- Keys are assumed to come from a total order.
  - Reflexive property: $x \leq x$
  - Antisymmetric property: $x \leq y \ \& \ y \leq x \Rightarrow x = y$
  - Transitive property: $x \leq y \ \& \ y \leq z \Rightarrow x \leq z$
- New operations:
  - closestKeyBefore(k): return key of item with largest key <= k
  - closestElemBefore(k): return element of item with largest key <= k
  - closestKeyAfter(k): return key of item with smallest key >= k
  - closestElemAfter(k): return element of item with smallest key >= k

Lookup Table

- A lookup table is a dictionary implemented by means of a sorted sequence
  - We store the items of the dictionary in an array-based sequence, sorted by key
  - We use an external comparator for the keys
- Performance:
  - findElement takes $O(n)$ time
  - insertItem takes $O(n)$ time since we have to shift items to make room for the new item
  - removeElement take $O(n)$ time since we have to shift items to compact the items after the removal
- The lookup table is effective only for dictionaries of small size or for dictionaries on which searches are the most common operations, while insertions and removals are rarely performed

Binary Search

- Binary search performs operation findElement(k) on a dictionary implemented by means of an *array-based sequence*, sorted by key
  - at each step, the number of candidate items is halved
  - terminates after O(log n) steps

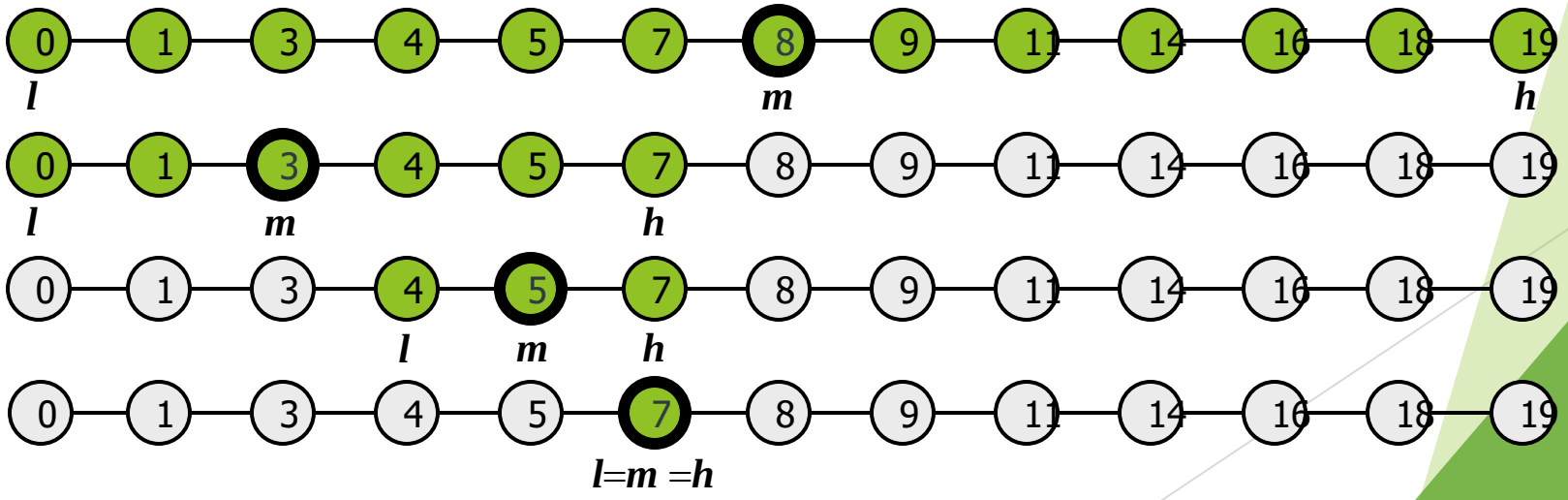
```
Algorithm BinarySearch(S, k, low, high)
    if low > high then
        return NO_SUCH_KEY
    else
        mid ← floor((low+high)/2)
        if k = key(mid) then
            return elem(mid)
        else if k < key(mid) then
            return BinarySearch(S,k,low,mid-1)
        else
            return BinarySearch(S,k,mid+1,high)
```

Call BinarySearch(S,k,0,n-1)

Binary Search

- Binary search performs operation findElement(k) on a dictionary implemented by means of an *array-based sequence*, sorted by key
  - at each step, the number of candidate items is halved
  - terminates after O(log n) steps

- Example: findElement(7)

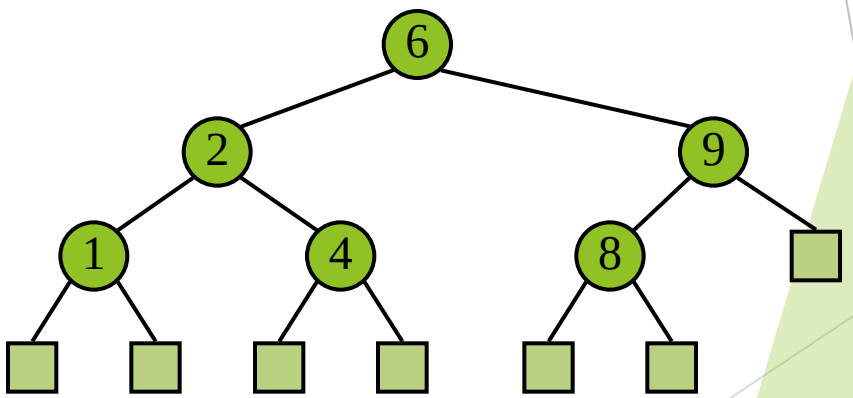


Lookup Table with Binary Search

- A lookup table is a dictionary implemented by means of a sorted sequence
  - We store the items of the dictionary in an array-based sequence, sorted by key
  - We use an external comparator for the keys
- Performance:
  - findElement takes $O(\log n)$ time, using binary search
  - insertItem takes $O(n)$ time since in the worst case we have to shift $n/2$ items to make room for the new item
  - removeElement take $O(n)$ time since in the worst case we have to shift $n/2$ items to compact the items after the removal
- The lookup table is effective only for dictionaries of small size or for dictionaries on which searches are the most common operations, while insertions and removals are rarely performed

Binary Search Tree

- A binary search tree is a binary tree storing keys (or key-element pairs) at its internal nodes and satisfying the following property:
  - Let u, v, and w be three nodes such that u is in the left subtree of v and w is in the right subtree of v. We have $key(u) \leq key(v) \leq key(w)$
- External nodes do not store items
- An inorder traversal of a binary search trees visits the keys in increasing order

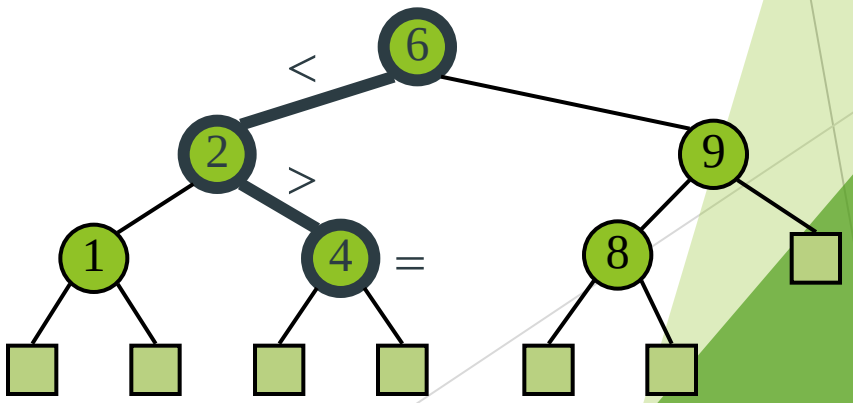


Search

- To search for a key k, we trace a downward path starting at the root
- The next node visited depends on the outcome of the comparison of k with the key of the current node
- If we reach a leaf, the key is not found and we return NO_SUCH_KEY
- Example: findElement(4)

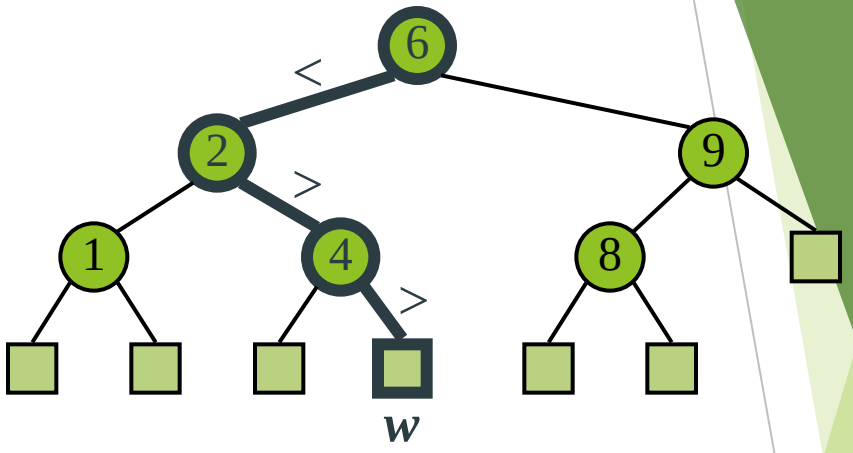
  findElement(3) ?

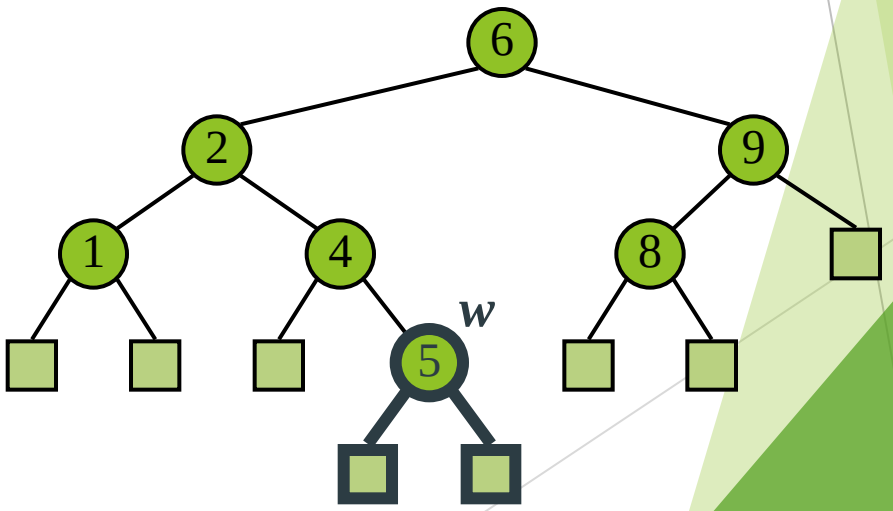
```
Algorithm findElement(k, v)
    if T.isExternal (v)
        return NO_SUCH_KEY
    if k < key(v)
        return findElement(k, T.leftChild(v))
    else if k = key(v)
        return element(v)
    else { k > key(v) }
        return findElement(k, T.rightChild(v))
```



Insertion

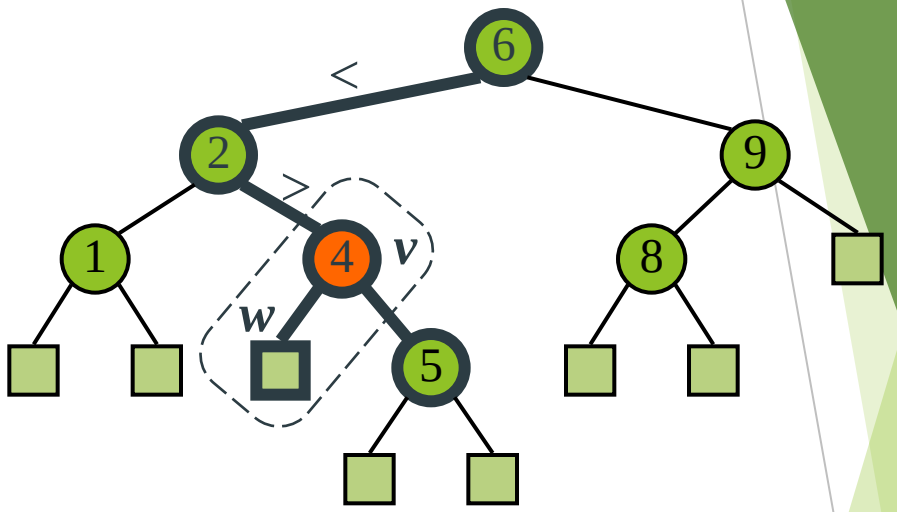
- To perform operation insertItem(k, o), we search for key k
- Assume k is not already in the tree, and let w be the leaf reached by the search
- We insert k at node w and expand w into an internal node
- Example: insert 5
- What if k is already in the tree?
  - Search in the right subtree of k for the next available external node following k.
  - Example: insert 6

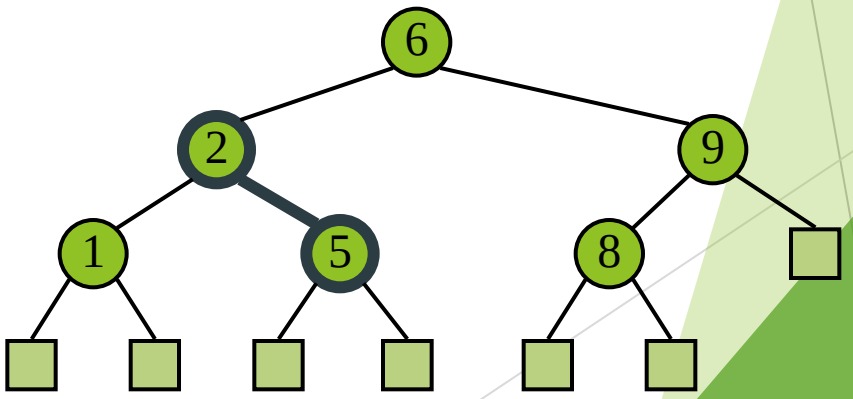




Deletion

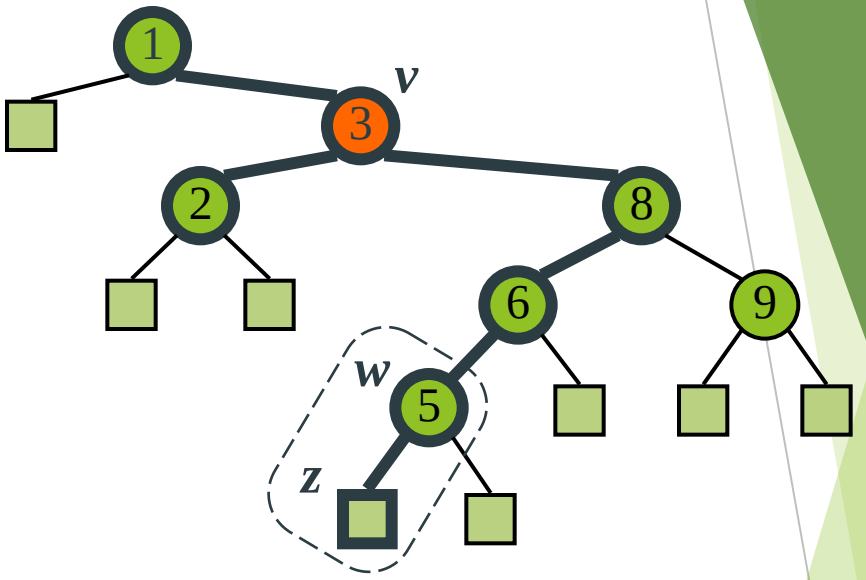
- To perform operation removeElement(k), we search for key k
- Assume key k is in the tree, and let v be the node storing k
- If node v has an external child w, we remove v and w from the tree with operation removeAboveExternal(w)
- Example: remove 4

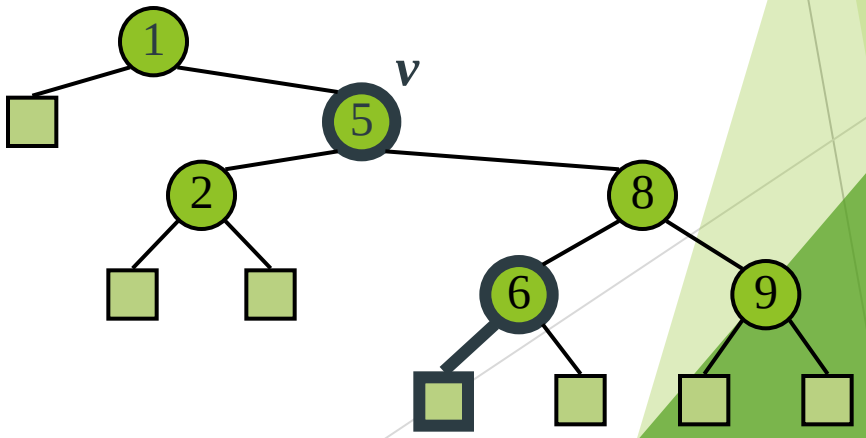




Deletion

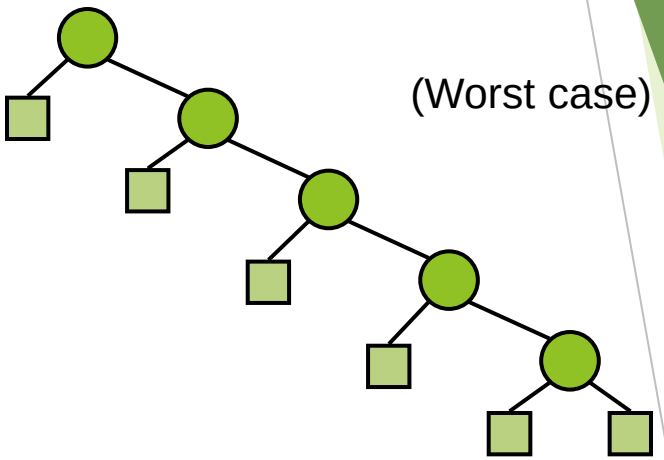
- We consider the case where the key k to be removed is stored at a node v whose children are both internal
  - we find the internal node w that follows v in an inorder traversal (the left-most internal node in the right sub-tree of v)
  - we copy $key(w)$ into node v
  - we remove node w and its left child z (which must be a leaf) by means of operation removeAboveExternal(z)
- Example: remove 3





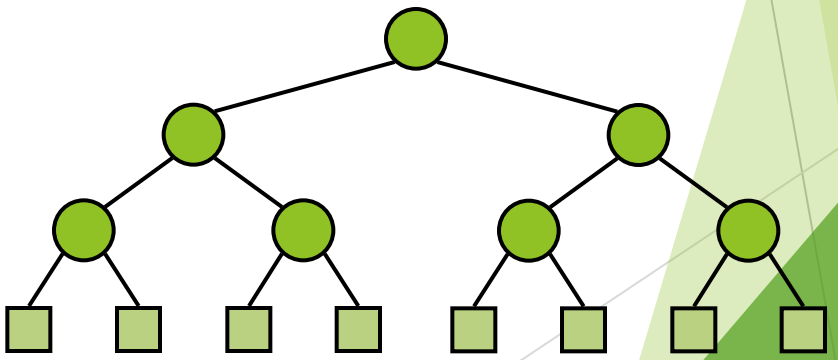
Performance

- Consider a dictionary with n items implemented by means of a binary search tree of height h
  - the space used is $O(n)$
  - methods findElement , insertItem and removeElement take $O(h)$ time



(Worst case)

- The height h is $O(n)$ in the worst case and $O(\log n)$ in the best case



(Best case)